

Inverse Problems in Geophysics

Traveltime tomography

2. MGPY+MGIN

Thomas Günther

thomas.guenther@geophysik.tu-freiberg.de

Tasks for today

- generate measuring scheme (`createCrossholeData()` @ `tt_utils.py`)
- generate a model with some anomalies using [pyGIMLi example](#)
- compute synthetic data and plot it (also as apparent velocity)
- assume absolute error and add random noise to the data
- generate rectangular model, compute SVD and plot singular values
- compute TSV inverse models for varying pseudo-rank p
- solve damped normal equations for a range of λ
- create smoothness matrix & solve smoothness-constrained inversion
- plot data misfit & model norm depending on p , play w model, noise

Code pieces

See also [pyGIMLi example](#)

```
1 import pygimli as pg
2 import pygimli.physics.traveltime as tt
3
4 data = tt.createCrossholeData()
5 grid = pg.createGrid(x=..., y=...)
6 grid.show()
7 velocity = ... grid.cellCount()
8 # create anomalies
9 grid.show(velocity)
10 data = tt.simulate(grid, data, slowness=..., secNodes=5)
11 pg.viewer.mpl.showDataContainerAsMatrix(data, "s", "g", "t");
```

Manager way

```
1 mgr = tt.TravelTimeManager(data, secNodes=10)
2 mgr.setMesh(grid, secNodes=10)
3 mgr.invert(maxIter=0, startModel=1);
4 J = mgr.fop.jacobian()
5 G = pg.utils.sparseMatrix2csr(J).todense()
```

Forward operator way

```
1 f = tt.
```

```
1 data = tt.DataContainer()
2 [data.createSensor([x1, yi]) for yi in sy] # left
3 [data.createSensor([x2, yi]) for yi in sy] # right
4 one = np.ones_like(sy, dtype=int) # 1 1 1
5 up = np.arange(len(sy)) # 1 2 .. 10
6 shot = []
7 geop = []
8 for i in range(len(sy)):
9     shot.extend(one * i)
10    geop.extend(up+len(sy))
11
12 data.resize(len(shot))
13 data["s"] = np.array(shot)
14 data["g"] = np.array(geop)
```